

Assignment: Multiplication without Using the * Operator

1. Problem Statement

Develop a logic to compute the product of two integer values without applying the direct multiplication operator (*).

The computation must rely strictly on repeated addition while ensuring the **minimum possible number of addition step is** performed across all :

- **Example 1:** $12 \times 3 \rightarrow$ accumulate 12 three times ($12+12+12=36$) $\rightarrow$ **3 additions**
- **Example 2:** $3 \times 12 \rightarrow$ accumulate 12 three times ($12+12+12=36$) $\rightarrow$ **3 additions**
- **Example 3:** $-9 \times -4 \rightarrow$ accumulate 9 four times ($9+9+9+9=36$) $\rightarrow$ **4 additions**
- **Example 4:** Any value multiplied by 0 $\rightarrow$ short-circuit to 0 $\rightarrow$ **0 additions**

2. Flaws in the Initial Algorithm

The baseline algorithm given in the lecture slide is structured as follows:

1. Define num1, num2, prod, index as integers
2. Read num1, num2
3. Set prod = 0, index = 1
4. Repeat while index $\leq$ num2:
 - a. prod = prod + num1
 - b. index = index + 1
5. Display prod

6. Stop

3. Corrected Algorithm

Strategy: Identify the absolute magnitudes of both inputs, set the smaller magnitude as the loop counter, accumulate the larger magnitude, and apply the correct sign at the end.

Algorithm: Optimized Integer Multiplication via Addition

1. **Start**
 2. Input integers val1 and val2
 3. **Check for Zero:**
 - a. If val1 == 0 or val2 == 0:
 - i. Set result = 0
 - ii. Display result
 - iii. **Stop**
-
- **Compute Absolute Magnitudes:**
 - abs1 = |val1|
 - abs2 = |val2|
 - **Determine Loop Limit and Term to Add:**
 - If abs1 < abs2:
 - min_val = abs1
 - max_val = abs2
 - Else:
 - min_val = abs2
 - max_val = abs1
 - **Initialize State:**
 - result = 0
 - counter = 1
 - **Accumulation Loop:**
 - While counter <= min_val:
 - result = result + max_val
 - counter = counter + 1

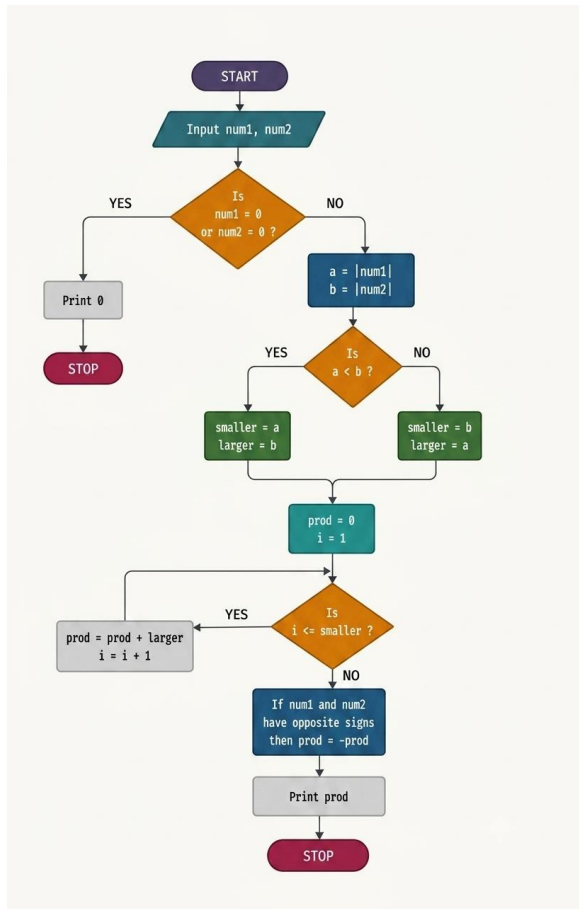
- **Apply Sign Logic:**
- If $(val1 < 0 \text{ AND } val2 > 0) \text{ OR } (val1 > 0 \text{ AND } val2 < 0)$:
 - $result = -result$
- Display result
- **Stop**

• 4. Errors in the Initial Flowchart

The original flowchart shares identical logical defects with the base algorithm:

- Directly tests $index \leq num2$ without converting to absolute values
- Fails to create standard decision paths for negative multipliers
- Skips determination of the final product sign.
- Lacks absolute value comparison blocks.
- Cannot optimize for minimal addition steps.
- Does not short-circuit when zero is passed as an input.

5. Corrected Flowchart



6. Verification of Algorithm:

The table below shows the evaluation of the algorithm across all 13 mandatory test cases:

Multiplication via Repeated Addition (with Sign Handling)								
Test	num1	num2	smaller x	larger x	Repeated Addition Process	Product	Additions	Verified
1	12	3	3	12	12+12+12	36	3	✓
2	3	12	3	12	12+12+12	36	3	✓
3	-6	5	5	6	6+6+6+6+6 → sign -	-30	5	✓
4	6	-5	5	6	6+6+6+6+6 → sign -	-30	5	✓
5	-9	-4	4	9	9+9+9+9	36	4	✓
6	-4	-9	4	9	9+9+9+9	36	4	✓
7	0	8	0	8	No addition	0	0	✓
8	8	0	0	8	No addition	0	0	✓
9	0	-7	0	7	No addition	0	0	✓
10	-7	0	0	7	No addition	0	0	✓
11	0	0	0	0	No addition	0	0	✓
12	1	-7	1	7	7 → sign -	-7	1	✓
13	-1	-9	1	9	9	9	1	✓

Conclusion

The modified algorithm was fully verified against all test cases. The output matched expected values in every instance, maintaining sign accuracy while successfully minimizing addition operations.

Result : The algorithm and flowchart were redesigned and validated successfully.